AGENTIC TORONTO TRAVEL ASSISTANT

Project Report | Tool-Using RAG System

Prepared: 23 October 2026

Deployment status: Local implementation prepared; public deployment pending.

Project type: Retrieval-Augmented Generation (RAG) with a tool-using agent backend

Prepared: 23 October 2026

Deployment status: Local implementation prepared; public deployment pending

Planned hosting: Streamlit Community Cloud (after repository publication and owner authorization)

1. Abstract

The Agentic Toronto Travel Assistant is an educational question-answering application built around a local travel-guide document. It combines semantic retrieval with a selectable language-model provider. Rather than sending each question directly to an LLM, the backend routes the request to a task type, retrieves relevant guide passages, checks whether the retrieved context is adequate, invokes an LLM when configured, and records an execution trace. An offline demo mode allows the retrieval and routing path to be demonstrated without making an LLM API call.

2. Problem statement

A plain LLM chatbot may answer confidently even when the supplied knowledge source does not contain the answer. A basic RAG application improves this by retrieving relevant source text, but a user may still have little visibility into the steps taken. This project adds a lightweight agent orchestration layer that selects a task pattern, invokes retrieval and validation tools, and exposes its trace for inspection.

3. Objectives

• Answer questions about Toronto using a supplied travel guide.

• Retrieve relevant guide passages with semantic embeddings.

• Route questions to suitable task patterns, including itinerary planning, food recommendations, transport advice, comparisons, and general Q&A.

• Avoid generating an answer when retrieval evidence is weak.

• Support OpenRouter, Google Gemini API, and local Ollama as answer-generation providers.

• Expose agent steps, retrieved chunk IDs, and similarity scores in the interface.

• Provide an offline demonstration path for the tool workflow.

4. System architecture

The application follows this flow:

1. User interface: Streamlit collects the question and settings.

2. Intent router: deterministic keyword rules select the task pattern.

3. Retrieval tool: Sentence Transformers encodes the query; cosine similarity against precomputed chunk embeddings ranks document passages.

4. Context validator: checks whether the top retrieval score passes a lightweight threshold. If evidence is weak, the agent returns a limitation message instead of calling the LLM.

5. Generation tool: the selected provider produces a concise answer based on the retrieved passages.

6. Response validator: checks for an empty response and retries once with stricter instructions if needed.

7. Trace output: the UI shows the route, tools, validation outcome, and retrieved source chunks.

This is a small educational agent architecture. It uses deterministic routing rather than an autonomous LLM tool-calling protocol, and its validation is intentionally lightweight. A non-empty answer and similarity threshold do not prove factual correctness.

5. Tools and technologies

• Python 3.10 or later

• Streamlit for the user interface

• Sentence Transformers (all-MiniLM-L6-v2) for local embeddings

• NumPy for similarity ranking

• Requests for provider APIs

• python-dotenv for local environment variables

• OpenRouter, Gemini API, or Ollama for optional answer generation

6. Knowledge source

The supplied project document is src/materials/torontoTravelAssistant.txt, with a corresponding Word document. It covers attractions, food and dining, accommodation, sample itineraries, seasonal events, transportation, neighborhoods, outdoor activities, and FAQs. The current project indexes this local guide at startup. This is a small curated knowledge source, not a comprehensive or live travel database. Opening times, prices, and transit information may change and should be independently verified before travel.

7. Agent tools

7.1 Intent routing

The router classifies questions into itinerary_planning, option_comparison, food_recommendation, transport_advice, or guide_qa. The routing is rule-based, making its behavior easy to inspect and reproduce.

7.2 Semantic retrieval

The guide is split into overlapping text chunks. The embedding model creates normalized vectors for each chunk. At question time, the query is embedded and ranked by dot product, which corresponds to cosine similarity for normalized vectors. The top-k passages are supplied as context to the selected LLM.

7.3 Context validation

The validator checks whether the retrieval result is non-empty and whether the best similarity score is at least 0.20. This threshold is a simple heuristic, not a calibrated confidence score; it may need adjustment after evaluation on representative questions.

7.4 Answer generation and response check

The selected provider receives instructions to answer only from retrieved evidence and acknowledge missing information. If generation returns an empty answer, the backend retries once with stricter instructions. The system does not implement a full semantic entailment checker, so factual review remains necessary.

8. User workflow

1. Start the Streamlit application.

2. Select OpenRouter, Gemini, or Ollama in the sidebar.

3. Configure the provider key or local model as appropriate.

4. Enter a travel question or choose a sample prompt.

5. The agent routes the request and retrieves relevant guide passages.

6. The app displays the answer and, when enabled, the execution trace and source excerpts.

7. Offline demo mode can be used to demonstrate routing and retrieval without an LLM API call.

9. Testing and verification

The Python files should be checked with python -m compileall src before demonstration. Manual functional tests should include:

• an attraction question, such as What can I visit in Toronto?;

• an itinerary question, such as Create a 3-day itinerary for Toronto.;

• a food question, such as Compare food areas in Toronto.;

• a transport question, such as How can I get around Toronto?;

• an unrelated question, to observe whether context validation declines to answer;

• offline demo mode, to verify that routing and retrieval run without an LLM provider call;

• each hosted/local provider separately, after its configuration is complete.

No live provider calls or public deployment are claimed as completed by this report. Provider integration must be tested with the user's own configured account, and free quotas may change.

10. Deployment location and date

Actual deployment status on 23 October 2026: not publicly deployed. The code is prepared to run locally with Streamlit, normally at http://localhost:8501. This is a local development address, not a public website.

Planned public host: Streamlit Community Cloud. Deployment requires publishing the project to a GitHub repository, connecting that repository to Streamlit Community Cloud, configuring secrets securely, and completing a successful build. Public URL: pending. Actual deployment date: pending; it must be entered only after deployment succeeds. This report intentionally does not invent a live URL or claim that hosting has already occurred.

For cloud deployment, use a hosted LLM provider unless Ollama is separately deployed to a secured, reachable server. A cloud-hosted app cannot normally access Ollama running only on the developer's own laptop.

11. Security and privacy

API keys should be stored in .env locally and in the host's secrets manager for deployment. The .env file must never be committed to a repository. The project archive excludes Git history and local environment secrets. Do not expose provider keys in screenshots, the demo, or the report.

12. Limitations

• The knowledge base is a static Toronto guide and can become outdated.

• Intent routing is based on keywords and may misclassify ambiguous questions.

• The retrieval threshold is heuristic and has not been calibrated on a labeled evaluation set.

• The response validator checks emptiness, not factual entailment.

• Model availability, API pricing, and free-tier quotas can change.

• Ollama requires local compute resources and is not automatically available to a cloud-hosted app.

13. Future improvements

• Add a benchmark of representative questions with retrieval precision and answer-grounding review.

• Replace keyword routing with structured intent classification and explicit tool schemas.

• Add citations with document chunk identifiers to every generated claim.

• Add a stronger answer-evidence verification stage and bounded retry policy.

• Add more source documents and a document-upload interface.

• Log latency and token usage while avoiding storage of sensitive user prompts.

• Publish the app and record the verified URL and actual deployment date.

14. Conclusion

The project demonstrates a practical progression from a simple RAG chatbot to a lightweight tool-using agent. The key contribution is the explicit backend orchestration of intent routing, semantic retrieval, evidence checking, optional LLM synthesis, and trace visibility. The project is ready for local testing and demonstration. Public deployment remains a separate step that requires repository and hosting authorization.